Jarosław Kuchta

Jakość Systemów Informatycznych

Jakość Oprogramowania

Pomiary w inżynierii oprogramowania

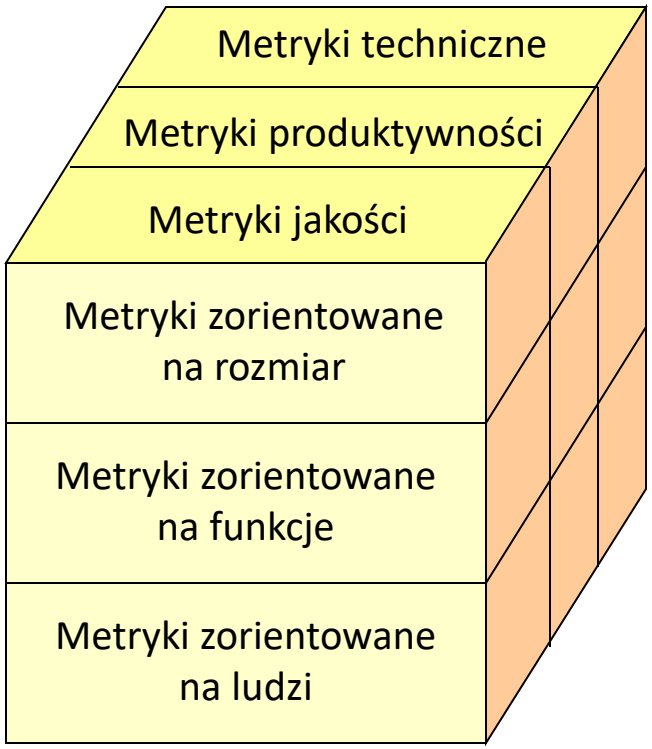
Cel pomiarów

- ocena jakości produktu
- ocena procesów (produktywności ludzi)
  - stworzenie podstawy dla szacowania
- ocena korzyści (nowe techniki i narzędzia)
  - ocena potrzeby nowych narzędzi lub szkoleń

Kategorie pomiarów

- pomiary bezpośrednie (np. długość, czas)
- pomiary pośrednie

Kategorie metryk w inżynierii oprogramowania



- Metryki techniczne
  - złożoność, modularność
- Metryki jakości
  - spełnienie wymagań użytkownika
- Metryki produktywności
  - wydajność procesu wytwarzania

- Metryki zorientowane na rozmiar
  - odnoszą się do rozmiaru kodu
- Metryki zorientowane na funkcje
  - odnoszą się do liczby funkcji
- Metryki zorientowane na ludzi
  - odnoszą się do pracy ludzkiej

Metryki zorientowane na rozmiar (1)

- Metryki bezpośrednie
  - rozmiar kodu [KLOC]
  - rozmiar dokumentacji [strony]
  - pracochłonność [osobomiesiące]
  - koszt
  - liczba defektów

Metryki zorientowane na rozmiar (2)

- Metryki pośrednie
  - produktywność
    = rozmiar kodu/pracochłonność
  - awaryjność
    = liczba defektów/rozmiar kodu
  - kosztowność
    = koszt/rozmiar kodu
  - udokumentowanie
    = rozmiar dokumentacji/ rozmiar kodu

Metryki zorientowane na rozmiar (za i przeciw)

- Za
  - rozmiar kodu może być łatwo policzony
  - rozmiar kodu jest używany w wielu modelach szacowania oprogramowania
  - wpływ rozmiaru kodu na inne czynniki jest dobrze udokumentowany
- Przeciw
  - rozmiar kodu jest zależny od języka programowania
  - zwięzłe, krótkie programy mają gorsze wskaźniki
  - nie nadają się dla języków nieproceduralnych
  - szacowanie rozmiaru kodu jest konieczne przed rozpoczęciem kodowania!

Metryki zorientowane na funkcje

- punkty funkcyjne (FP – Function Points)
- punkty funkcjonalne (FP – Feature Points)

Punkty funkcyjne (1)

Parametr pomiarowy	Liczba	Współczynnik wagowy			Liczba ważona
		Prosty	Średni	Złożony	
Liczba wejść od użytkownika		× 3	4	6 =	
Liczba wyjść do użytkownika		× 4	5	7 =	
Liczba zapytań		× 3	4	6 =	
Liczba plików		× 7	10	15 =	
Liczba interfejsów zewnętrznych		× 5	7	10 =	
Liczba punktów					

Punkty funkcyjne (2)

F_i:	0	1	2	3	4	5
	brak wpływu	incydentalnie	umiarkowanie	średnio	znacząco	zasadniczo

1. Czy system wymaga wiarygodnego zachowywania i odzyskiwania danych?
2. Czy wymagane jest przekazywanie danych?
3. Czy występują funkcje przetwarzania rozproszonego?
4. Czy wydajność jest krytyczna?
5. Czy system ma pracować w istniejącym, trudnym środowisku operacyjnym?
6. Czy system wymaga wprowadzania danych *on-line*?
7. Czy dane wprowadzane *on-line* wymagają transakcji wejściowych zbudowanych na wielu ekranach lub operacjach?
8. Czy główne pliki są aktualizowane *on-line*?
9. Czy wejścia, wyjścia, pliki lub interakcje są złożone?
10. Czy wewnętrzne przetwarzanie jest złożone?
11. Czy kod jest zaprojektowany do powtórnego wykorzystania?
12. Czy konwersja i instalacja jest zawarta w projekcie?
13. Czy system został zaprojektowany dla wielu instalacji w różnych organizacjach?
14. Czy aplikacja jest zaprojektowana w sposób przyjazny dla użytkownika i tak, by ułatwiać wprowadzanie zmian?

Punkty funkcyjne (3)

$$FP = \text{liczba punktów} \times [0{,}65 + 0{,}01 \times \text{Sum}(F_i)]$$

- Metryki pośrednie
  - produktywność = FP/pracochłonność
  - awaryjność = liczba defektów/FP
  - kosztowność = koszt/FP
  - udokumentowanie = rozmiar dokumentacji/FP

Punkty funkcjonalne

Parametr pomiarowy	Liczba	Waga			Liczba ważona
Liczba wejść od użytkownika		×	4	=	
Liczba wyjść do użytkownika		×	5	=	
Liczba interakcji z użytkownikiem		×	4	=	
Liczba plików		×	7	=	
Liczba interfejsów zewnętrznych		×	7	=	
Algorytmy		×	3	=	
Liczba punktów					

Punkty funkcyjne/ funkcjonalne (za i przeciw)

- Za
  - są niezależne od języka programowania
  - nadają się zarówno dla języków proceduralnych jak i nieproceduralnych
  - mogą być stosowane we wczesnych fazach planowania
- Przeciw
  - obliczenia mają charakter częściowo subiektywny (zwłaszcza p. funkcyjne)
  - dane są trudne do zebrania, trudniejsze niż rozmiar kodu
  - nie mają bezpośredniego znaczenia fizycznego

Zależność LOC/FP dla różnych języków programowania

Język programowania	LOC/FP
Asembler	300
COBOL	100
FORTRAN	100
PASCAL	90
ADA	70
Języki obiektowe	30
Języki czwartej generacji	20
Generatory kodu	15

Metryki złożoności

- metryka Halsteada
- metryka cyklometryczna McCabe’a

Metryki Halsteada (1)

n_1 – liczba różnych operatorów w programie

n_2 – liczba różnych operandów w programie

N_1 – całkowita liczba operatorów

N_2 – całkowita liczba operandów

Metryki Halsteada – przykład (1)

```
Sub Sort(X,N)
  Dim X(N)
  If N<2 Return
  For I = 2 To N
    For J = 1 To I
      IF X(I)<X(J) Then
        Save = X(I)
        X(I) = X(J)
        X(J) = Save
      End If
    Next
  Next
End Sub
```

Lp	Operator	Liczba
1	koniec instrukcji	7
2	indeksowanie	6
3	=	5
4	IF	2
5	FOR	2
6	,	2
7	<	2
8	RETURN	1
9	koniec programu	1
$n_1=9$		$N_1=28$

Metryki Halsteada – przykład (2)

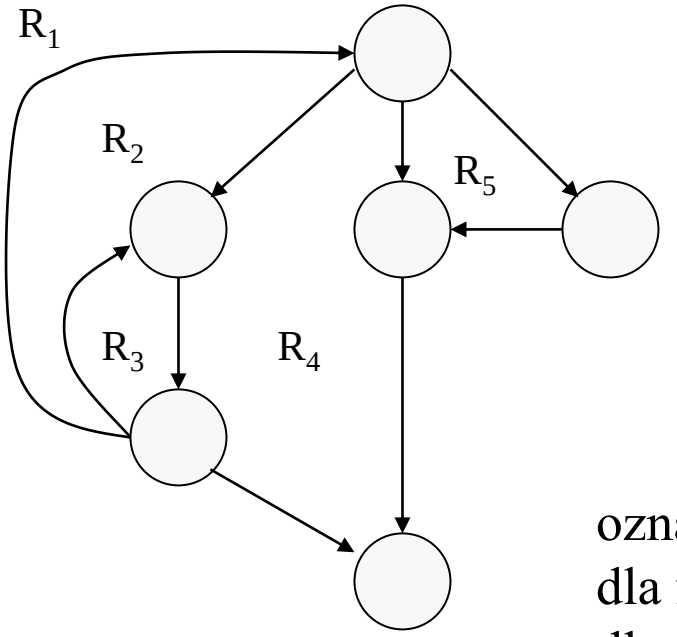
```
Sub Sort(X,N)
  Dim X(N)
  If N<2 Return
  For I = 2 To N
    For J = 1 To I
      IF X(I)<X(J) Then
        Save = X(I)
        X(I) = X(J)
        X(J) = Save
      End If
    Next
  Next
End Sub
```

Lp	Operand	Liczba
1	X	6
2	I	5
3	J	4
4	N	2
5	2	2
6	Save	2
7	1	1
n_2=7		N_2=22

Metryki Halsteada (3)

- długość programu: $N = N1 + N2$
- rozmiar słownika: $n = n_1 + n_2$
- objętość algorytmu: $V = N \log_2(n)$
  - stosowana zamiast LOC
  - objętość funkcji powinna być od 20 do 1000
  - objętość pliku powinna być od 100 do 8000
- poziom trudności: $D = (n_1/2)*(N_2/n_2)$
  - wyznacza stopień odporności na błędy
- poziom programu: $L = 1/D$
- wysiłek implementacyjny: $E = V*D$
- czas na implementację: $T = E/18$ *(w sekundach)*
- liczba potencjalnych błędów: $B = E^{(2/3)} / 3000$

Metryka złożoności cyklometrycznej McCabe'a



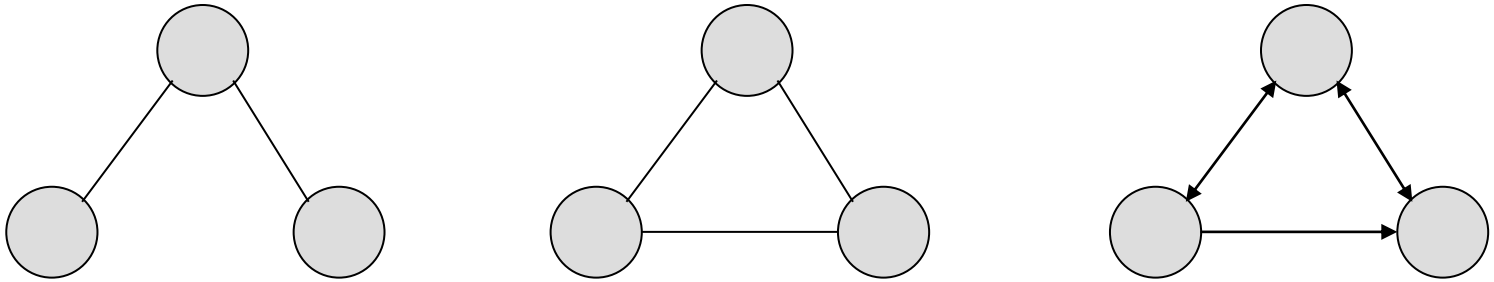
v(G) = 5

oznacza liczbę potencjalnych ścieżek wykonania
dla funkcji powinna nie większa niż 15
dla plików powinna nie większa niż 100

Maintainability Index = MAX(0,(171 - 5.2 * ln(Halstead Volume) - 0.23 * (Cyclomatic Complexity) - 16.2 * ln(Lines of Code))*100 / 171)

Metryki spójności grafów

- Spójność grafu
  - spójność słaba
  - nierozdzielność (węzłowa, krawędziowa)
  - spójność silna



Ankiety (kwestionariusze)

- Brak metryk obiektywnych
- Duża subiektywność
- Wymuszenie obiektywności – pytania tak/nie
- Duża liczba pytań
  - niechęć do odpowiedzi
  - nierzetelność odpowiedzi
- Wiarygodność oceny
- Duża liczba oceniających

Bibliografia

- Pressman R.S., *Software engineering. A practitioner's approach*, McGraw-Hill, International Edition, 1992
- Halstead Maurice, *Elements of Software Science*, Elsevier Science Ltd, 1977
- http://www-ivs.cs.uni-magdeburg.de/sw-eng/us/experiments/hals/
- http://yunus.hacettepe.edu.tr/~sencer/complexity.html